

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente "CUNOC"
División Ciencias de la Ingeniería
Laboratorio de Lenguajes Formales y de Programación
Ing. Daniel Gonzalez



Manual Técnico PromptZal v2. - Analizador Léxico

William Eduardo Gómez Sánchez
Carné: 202232064

Guatemala, Quetzaltenango, 17 de septiembre de 2026

Introducción

El presente documento describe los aspectos técnicos del analizador léxico desarrollado para el lenguaje PromptZal, como parte del Proyecto 1 del Laboratorio de Lenguajes Formales y de Programación.

PromptZal es un lenguaje diseñado para que los programadores puedan redactar prompts estructurados de forma rápida, los cuales son utilizados posteriormente en herramientas de inteligencia artificial.

Este proyecto parte del tokenizador construido en la Práctica 1, el cual funcionaba de forma exploratoria, sin un fundamento formal detrás. En esta entrega, el analizador se eleva a un componente profesional: su reconocimiento está respaldado por un autómata finito determinista (AFD) diseñado explícitamente para cada categoría de token, expresado también como una gramática regular equivalente. El analizador sigue siendo de implementación manual, recorriendo el texto de entrada carácter por carácter, ahora guiado por el AFD formalizado. Además, se incorpora recuperación de errores léxicos cuidada, una interfaz gráfica desarrollada en Java Swing para operar el sistema, y la generación de reportes en HTML junto con la graficación del autómata mediante Graphviz.

Requisitos del Sistema

Requisitos mínimos para ejecutar el proyecto:

- Sistema Operativo: Windows, Linux o macOS
- Java Development Kit (JDK): Versión 17 o superior
- Memoria RAM: 512 MB mínimo
- Espacio en disco: 50 MB mínimo
- Un archivo de texto con extensión .pz para analizar

Para compilar el proyecto:

- Apache Maven 3.6 o superior
- NetBeans IDE 17 o superior (recomendado)

Para ejecutar el JAR:

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

Para compilar desde NetBeans:

1. Abrir el proyecto en NetBeans
2. Click derecho en el proyecto
3. Seleccionar "Clean and Build"
4. El JAR se genera en la carpeta target

Para compilar desde consola:

1. Abrir CMD o PowerShell
2. Navegar a la carpeta raíz del proyecto: `cd ruta\del\proyecto`
3. Ejecutar: `mvn clean package`
4. El JAR se genera en la carpeta target

Para ejecutar desde consola:

1. Navegar a la carpeta target:
`cd ruta\del\proyecto\target`

2. Ejecutar:

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

Descripción de Tecnologías

Lenguaje de programación:

- Java 17 — Lenguaje principal del proyecto, utilizado para implementar toda la lógica del analizador léxico, la interfaz gráfica y la generación de reportes.

Herramientas de desarrollo:

- NetBeans IDE 17 — Entorno de desarrollo utilizado para escribir, compilar y depurar el código.
- Apache Maven — Herramienta de gestión y construcción del proyecto, utilizada para compilar y generar el archivo JAR ejecutable.
- draw.io — Herramienta utilizada para elaborar el diagrama de clases del proyecto.
- Graphviz — Herramienta externa utilizada para graficar el autómata finito determinista (AFD) a partir del código DOT generado por el programa. Graphviz únicamente dibuja el grafo; no interviene en el reconocimiento de tokens.

Librería gráfica:

- Java Swing — Utilizada para construir la interfaz gráfica completa del proyecto: el editor de texto, los botones de operación (abrir, guardar, analizar, generar reportes, generar AFD), y los paneles de resultados (tokens y errores) mostrados mediante `JTable`.

Formato de salida:

- HTML y CSS — Utilizados para generar los reportes de tokens, errores léxicos y estadísticas, mostrados en tablas legibles dentro de un navegador web, con estilo visual propio consistente con el tema de la interfaz gráfica.

Características técnicas:

- Manejo de archivos — Se utiliza `Files.readAllBytes()` para leer el contenido completo del archivo `.pz`, y `FileWriter` para generar los reportes HTML y el código DOT del AFD.
- Colecciones — Se utiliza `List` (implementada con `ArrayList`) para almacenar los tokens y errores léxicos encontrados durante el análisis, ya que su cantidad no se conoce de antemano.
- Procesos externos — Se utiliza `ProcessBuilder` para invocar a Graphviz como proceso del sistema operativo y generar la imagen del AFD a partir del código DOT.

Conceptos de POO aplicados:

- Encapsulamiento — Los atributos de las clases del proyecto son privados, con acceso mediante getters y setters o métodos públicos específicos.
- Separación de responsabilidades (arquitectura por capas) — El proyecto se organiza en paquetes según su responsabilidad: lógica de análisis, generación de salidas, manejo de archivos, coordinación (controlador) y presentación (vista), evitando que una misma clase mezcle tareas de distinta naturaleza.
- Manejo de excepciones — Se definen excepciones propias (`ExcepcionArchivoNoEncontrado`, `ExcepcionGraphvizNoDisponible`) para reportar de forma clara y en español los errores que pueden ocurrir al leer archivos o al invocar Graphviz, en lugar de propagar las excepciones genéricas de Java sin contexto adicional.

Arquitectura del Proyecto

El proyecto está organizado en paquetes según su responsabilidad, siguiendo una separación clara entre la lógica de análisis, la generación de salidas, el manejo de archivos, la coordinación del flujo y la presentación:

- **app** → Contiene la clase principal `Promptzal`, con el método `main`, encargada únicamente de iniciar la interfaz gráfica.
- **modelo** → Contiene las clases de dominio `Token` y `ErrorLexico`, que representan respectivamente un componente léxico reconocido y un error encontrado durante el análisis.
- **logica** → Contiene la clase `AnalizadorLexico`, encargada exclusivamente del reconocimiento léxico del texto de entrada.
- **generadores** → Contiene las clases `GeneradorReporte` (reportes HTML de tokens, errores y estadísticas) y `GeneradorAFD` (código DOT del autómata y generación de su imagen mediante Graphviz).
- **archivos** → Contiene la clase `GestorArchivos`, encargada de leer y escribir archivos `.pz` desde el sistema de archivos.
- **controlador** → Contiene la clase `ControladorAnalisis`, encargada de coordinar el flujo completo entre la interfaz gráfica y el resto de las clases del backend.
- **excepciones** → Contiene las excepciones propias `ExcepcionArchivoNoEncontrado` y

`ExcepcionGraphvizNoDisponible`, utilizadas para reportar errores específicos del dominio con mensajes claros en español.

- **vista** → Contiene la clase `VentanaPrincipal`, que implementa la interfaz gráfica completa mediante Java Swing, y la clase `Colores`, que centraliza la paleta de colores utilizada en toda la aplicación.

Esta separación permite que cada clase tenga una única responsabilidad bien definida: el analizador léxico no conoce nada sobre archivos, reportes ni interfaz gráfica; los generadores no conocen nada sobre cómo se obtuvo el contenido que procesan; y es el controlador quien coordina la comunicación entre todas las piezas, sin que ninguna dependa directamente de la interfaz gráfica. **Esta organización responde al patrón MVC**

(Modelo-Vista-Controlador): el paquete `modelo` junto con `logica`, `generadores` y `archivos` conforman el modelo del sistema; `vista` contiene la interfaz gráfica; y `controlador` actúa como intermediario entre ambos, evitando que la vista dependa directamente de la lógica de negocio.

Descripción de Clases

Paquete app

- **Promptzal** — Clase principal del proyecto. Contiene únicamente el método `main`, que instancia la ventana principal de la interfaz gráfica y la muestra en pantalla. Toda la lógica de negocio se delega a las demás clases; esta clase no contiene ninguna decisión de flujo.

Paquete modelo

- **Token** — Clase que representa un componente léxico reconocido por el analizador. Almacena el número correlativo del token, su lexema (el texto exacto reconocido), su tipo (por ejemplo IDENTIFICADOR o ENTERO), y la fila y columna donde comienza en el archivo original.
- **ErrorLexico** — Clase que representa un error léxico detectado durante el análisis. Almacena el lexema o carácter problemático, una descripción textual del error (por ejemplo "Cadena sin cerrar"), y la fila y columna donde ocurrió.

Paquete logica

- **AnalizadorLexico** — Clase central del proyecto, encargada de recorrer el texto de entrada carácter por carácter y reconocer cada componente léxico según el autómata

finito determinista diseñado para el lenguaje. Al finalizar el análisis, expone dos listas: una con los tokens reconocidos y otra con los errores encontrados.

- **espiar()** — Permite consultar el carácter que sigue inmediatamente a la posición actual del recorrido, **sin** avanzar el puntero de lectura. Se utiliza para decidir, antes de consumir un carácter, si una transición del autómata debe tomarse o no; por ejemplo, para confirmar que después de un guion (-) viene un mayor-que (>) antes de reconocer la flecha, o que después de un punto viene un dígito antes de aceptarlo como parte de un número decimal.
- **avanzar()** — Mueve el puntero de lectura una posición hacia adelante y actualiza la fila y columna actuales en consecuencia: si el carácter que se deja atrás es un salto de línea, incrementa la fila y reinicia la columna a 1; en cualquier otro caso, simplemente incrementa la columna.
- **leerPalabra()** — Reconoce identificadores, palabras reservadas, comandos de IA y conectores de palabra. Acumula caracteres mientras sean letras, dígitos o guion bajo, y al terminar consulta listas predefinidas para clasificar el lexema resultante en su categoría correspondiente; si no coincide con ninguna lista, se clasifica como IDENTIFICADOR.
- **esCaracterDePalabra(c)** — Determina si un carácter puede formar parte de una palabra (letra, dígito o guion bajo). Es utilizado tanto por **leerPalabra()** como por **leerDirectiva()** para decidir cuándo continuar acumulando y cuándo detenerse.
- **clasificarPalabra(palabra)** — Recibe el lexema ya acumulado y determina su tipo consultando, en orden de prioridad, las listas de palabras reservadas, comandos de IA y conectores; si no aparece en ninguna, devuelve IDENTIFICADOR.
- **contiene(arreglo, valor)** — Método de apoyo que recorre un arreglo de cadenas y determina si contiene un valor específico. Se reutiliza en la clasificación de palabras y en la validación de directivas.
- **leerDirectiva()** — Reconoce las directivas del lenguaje (**@modelo**, **@rol**, **@formato**). Descarta el símbolo **@** inicial, acumula caracteres de palabra, y valida el lexema resultante contra la lista de directivas permitidas; si no es válido, reporta un error léxico en lugar de generar un token.
- **esDirectivaValida(directiva)** — Determina si el nombre de una directiva ya acumulada pertenece al conjunto de directivas reconocidas por el lenguaje.
- **leerCadena()** — Reconoce literales de cadena delimitados por comillas dobles. Descarta la comilla de apertura y acumula caracteres hasta encontrar la comilla de cierre; si se alcanza un salto de línea o el final del archivo antes de encontrar el cierre, reporta un error de "Cadena sin cerrar" con el contenido acumulado hasta ese punto.
- **leerNumero()** — Reconoce literales numéricos, tanto enteros como decimales. Acumula dígitos consecutivos y, opcionalmente, un único punto seguido de

más dígitos (verificado con `espíar()` antes de aceptarlo), clasificando el resultado como ENTERO o DECIMAL según corresponda.

- `leerComentario()` — Reconoce comentarios de línea (`//`) y de bloque (`/* */`), ninguno de los cuales genera token. Determina el tipo de comentario examinando el carácter siguiente a la primera diagonal; para el de línea, descarta hasta el salto de línea o el final del archivo; para el de bloque, descarta hasta encontrar el cierre `*/`, reportando un error léxico si el archivo termina sin encontrarlo.
- `leerConectorFlecha()` — Reconoce el conector `->`. Utiliza `espíar()` para confirmar que, tras un guion, sigue un signo mayor-que; si es así, genera el token CONECTOR; en caso contrario, reporta un error de carácter no reconocido.
- `leerSimboloSuelto(símbolo)` — Reconoce los símbolos de un solo carácter del lenguaje (`=`, `+`, `{`, `}`, `(`, `)`, `,`). Clasifica el símbolo como OPERADOR o DELIMITADOR según cuál haya sido, sin necesidad de acumulación ni de un paso adicional de validación.
- `analizar()` — Método principal que recorre el texto completo carácter por carácter. En cada posición, examina el carácter actual y delega el reconocimiento al método correspondiente según su naturaleza (letra, dígito, comilla, arroba, diagonal, guion, símbolo suelto, espacio); cualquier carácter que no encaje en ninguna categoría se reporta como error de "Carácter no reconocido", y el análisis continúa sin detenerse.

Paquete generadores

- **GeneradorReporte** — Clase encargada de generar los reportes de salida en formato HTML.
 - `generarReporteTokens(tokens, ruta)` — Construye una página HTML con una tabla que lista cada token reconocido (número, lexema, tipo, fila y columna), y la escribe en la ruta indicada.
 - `generarReporteErrores(errores, ruta)` — Construye una página HTML con una tabla que lista cada error léxico encontrado (lexema, descripción, fila y columna); si no hay errores, genera un mensaje indicándolo explícitamente en lugar de una tabla vacía.
 - `generarReporteEstadísticas(tokens, errores, ruta)` — Calcula y presenta en HTML el total de tokens, el total de líneas (a partir del número de fila más alto entre los tokens), el total de errores, y la frecuencia de cada tipo de token reconocido por el lenguaje.
 - `escribirArchivo(ruta, contenido)` — Método privado de apoyo que escribe una cadena de texto en un archivo dado, reutilizado por los tres métodos anteriores.
- **GeneradorAFD** — Clase encargada de la representación gráfica del autómata finito determinista del lenguaje.

- `generarCodigoDOT()` — Devuelve, como cadena de texto, el código en formato DOT que describe la totalidad del AFD diseñado para PromptZal (sus estados, transiciones y estados de aceptación o error para las siete categorías de token). Este código no depende del archivo `.pz` analizado; representa siempre el mismo autómata, el del lenguaje en sí.
- `generarImagen(rutaDot, rutaImagen)` — Escribe el código DOT en un archivo y, mediante `ProcessBuilder`, invoca a Graphviz como proceso externo del sistema operativo para generar la imagen del autómata; si Graphviz no está instalado o no se encuentra en el PATH del sistema, lanza `ExcepcionGraphvizNoDisponible`.

Paquete archivos

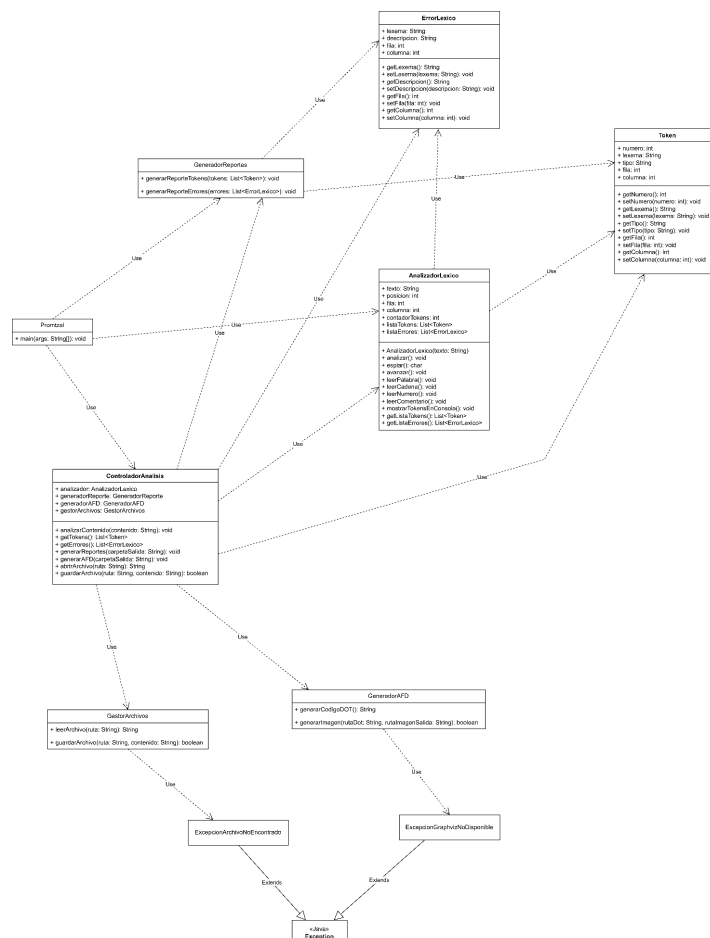
- **GestorArchivos** — Clase encargada de las operaciones de lectura y escritura de archivos `.pz`.
 - `leerArchivo(ruta)` — Lee el contenido completo de un archivo y lo devuelve como cadena de texto; si el archivo no existe o no puede leerse, lanza `ExcepcionArchivoNoEncontrado`.
 - `guardarArchivo(ruta, contenido)` — Escribe una cadena de texto en la ruta indicada, devolviendo un valor booleano que indica si la operación tuvo éxito.

Paquete controlador

- **ControladorAnalisis** — Clase que coordina el flujo completo del programa, actuando como intermediaria entre la interfaz gráfica y el resto del backend.
 - `analizarContenido(contenido)` — Crea una instancia de `AnalizadorLexico` con el contenido recibido y ejecuta el análisis léxico, almacenando internamente el resultado.
 - `getTokens()` / `getErrores()` — Devuelven las listas de tokens y errores obtenidas en el último análisis realizado.
 - `generarReportes(carpeta)` — Delega en `GeneradorReporte` la generación de los tres reportes HTML (tokens, errores y estadísticas).
 - `generarAFD(carpeta)` — Delega en `GeneradorAFD` la generación del código DOT y la imagen del autómata.
 - `abrirArchivo(ruta)` / `guardarArchivo(ruta, contenido)` — Delegan en `GestorArchivos` las operaciones de lectura y escritura de archivos `.pz`.

Paquete excepciones

- **ExcepcionArchivoNoEncontrado** — Excepción propia lanzada cuando un archivo `.pz` no puede ser localizado o leído, incluyendo en su mensaje la ruta que se intentó abrir.



Para visualizar el diagrama completo también se puede acceder al siguiente enlace:
<https://drive.google.com/file/d/1QoXpoViSqhsERFKxmMQ6SFdZEZYi6J2F/view?usp=sharing>

Lista de Tokens

El analizador léxico reconoce las siguientes categorías de tokens:

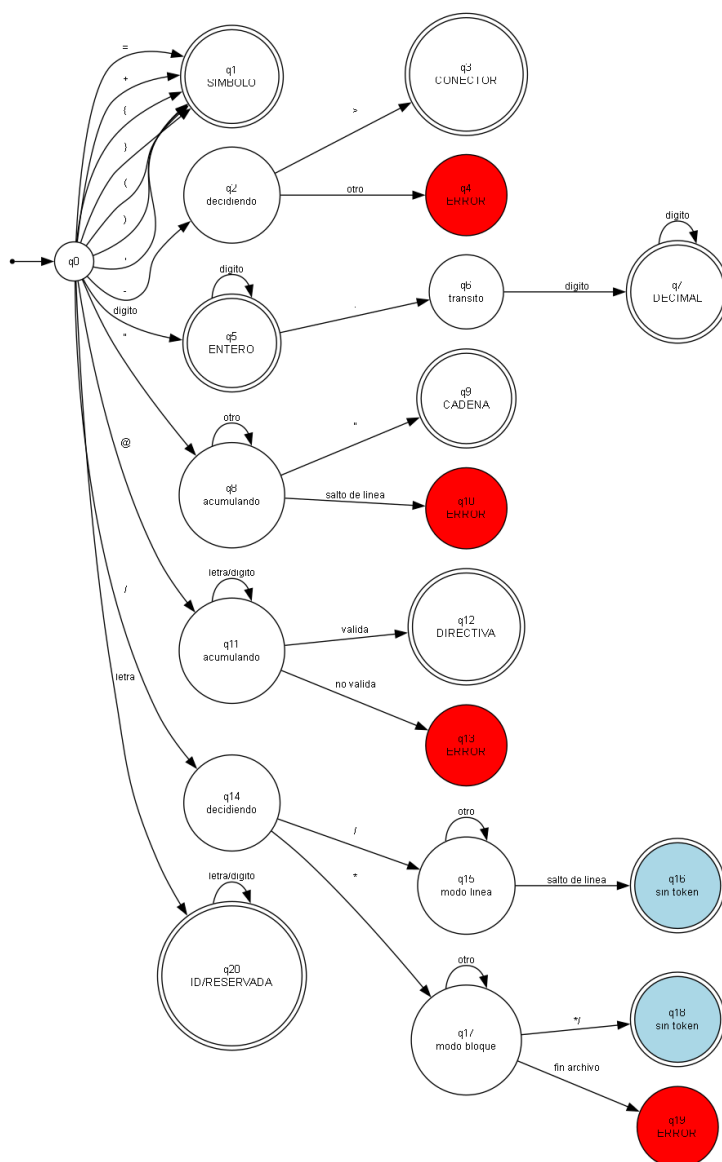
Categoría	Ejemplos	Tipo Asignado
Directivas	@modelo, @rol, @formato	DIRECTIVA
Palabras reservadas	AGENTE, contexto, variable, EJECUTAR, EXPORTAR	PALABRA_RESERVADA
Comandos de IA	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER, CARGAR	COMANDO_IA
Conectores	SOBRE, DESDE, EN, COMO, ->	CONECTOR
Identificadores	analista, ventas, resultado	IDENTIFICADOR
Literales de cadena	"texto entre comillas"	CADENA
Literales enteros	10, 100	ENTERO
Literales decimales	0.7, 3.14	DECIMAL
Operadores	=, +	OPERADOR
Delimitadores	{ } ()	DELIMITADOR
Comentarios	// línea, /* bloque */	(se ignoran, no generan token)

Autómata Finito Determinista y Gramática Regular

El reconocimiento léxico de PromptZal está formalmente respaldado por un autómata finito determinista (AFD) diseñado para el lenguaje completo, del cual se deriva una gramática regular equivalente. Ambos comparten un mismo punto de partida: un estado inicial único (q0), correspondiente al ciclo principal del método `analizar()`, del cual se ramifican siete caminos distintos según el primer carácter que distingue a cada categoría de token (letra, arroba, comilla, dígito, diagonal, guion, o símbolo suelto).

Convención utilizada en el diagrama:

- Círculo simple (blanco): estado de tránsito, aún no se ha reconocido un token completo.
- Doble círculo (blanco): estado de aceptación que genera un token.
- Doble círculo (celeste): estado de aceptación que no genera token (comentarios).
- Círculo (rojo): estado de error.



El AFD fue generado como imagen mediante Graphviz, a partir del código DOT emitido por el propio programa (clase `GeneradorAFD`, método `generarCodigoDOT()`), cumpliendo con el requerimiento de que sea el programa quien produzca la graficación del autómata.

A continuación se presenta la gramática regular equivalente para cada categoría de token, donde **l** representa una letra, **d** un dígito, y **c** cualquier carácter del alfabeto salvo el indicado explícitamente como excepción en cada caso.

Identificador / palabra reservada / comando / conector de palabra:

`<PALABRA>` $\rightarrow$ **l** `<PALABRA>`

`<PALABRA>` $\rightarrow$ **l**

Símbolo de un carácter:

`<SIMBOLO>` $\rightarrow$ = | + | { | } | (|) | ,

Conector flecha:

`<FLECHA>` $\rightarrow$ - >

Número (entero / decimal):

`<NUMERO>` $\rightarrow$ **d** `<NUMERO>`

`<NUMERO>` $\rightarrow$ **d**

`<NUMERO>` $\rightarrow$ **d** . `<FRAC>`

`<FRAC>` $\rightarrow$ **d** `<FRAC>`

`<FRAC>` $\rightarrow$ **d**

Cadena:

<CADENA> → " <CONTENIDO_CADENA>

<CONTENIDO_CADENA> → c <CONTENIDO_CADENA>

<CONTENIDO_CADENA> → "

Directiva:

<DIRECTIVA> → @ <CONTENIDO_DIRECTIVA>

<CONTENIDO_DIRECTIVA> → l <CONTENIDO_DIRECTIVA>

<CONTENIDO_DIRECTIVA> → l

Comentario de línea:

<COMENTARIO_LINEA> → / / <CONTENIDO_LINEA>

<CONTENIDO_LINEA> → c <CONTENIDO_LINEA>

<CONTENIDO_LINEA> → \n

Comentario de bloque:

<COMENTARIO_BLOQUE> → / * <CONTENIDO_BLOQUE>

<CONTENIDO_BLOQUE> → c <CONTENIDO_BLOQUE>

<CONTENIDO_BLOQUE> → * /

Cabe destacar que la clasificación fina de cada token (por ejemplo, distinguir una palabra reservada de un identificador, o un operador de un delimitador) no se resuelve dentro de la gramática misma, sino mediante una consulta a tablas predefinidas una vez reconocida la forma general del token — el mismo principio aplicado en la implementación del código, donde el autómata reconoce la forma y el programa decide el tipo específico.

Descripción del Recorrido

El analizador léxico recorre el texto de entrada de principio a fin, carácter por carácter, mediante un ciclo principal implementado en el método `analizar()`. Este recorrido está guiado por el autómata finito determinista diseñado para el lenguaje (ver sección "Autómata Finito Determinista"): cada rama del dispatcher corresponde al estado inicial (q0) tomando la transición correspondiente al primer carácter que distingue a cada categoría de token.

En cada posición, se examina el carácter actual y, según su naturaleza, se delega el reconocimiento al método correspondiente:

Si el carácter es una letra o guion bajo, se invoca el método `leerPalabra()`, que acumula caracteres mientras sean letras, dígitos o guiones bajos, y clasifica la palabra resultante como palabra reservada, comando de IA, conector o identificador, según listas predefinidas.

Si el carácter es un arroba (@), se invoca `leerDirectiva()`, que funciona de forma similar a `leerPalabra()` pero omitiendo el símbolo inicial, y valida el resultado contra la lista de directivas permitidas antes de generar el token.

Si el carácter es una comilla ("), se invoca `leerCadena()`, que acumula el contenido hasta encontrar la comilla de cierre. Si se alcanza un salto de línea o el final del archivo antes de encontrar el cierre, se reporta como error léxico.

Si el carácter es un dígito, se invoca `leerNumero()`, que acumula dígitos y, opcionalmente, un único punto decimal seguido de más dígitos (verificado mediante anticipación antes de aceptarlo), clasificando el resultado como entero o decimal.

Si el carácter es una diagonal (/), se invoca `leerComentario()`, que determina si se trata de un comentario de línea o de bloque, descartando su contenido sin generar ningún token. Si un comentario de bloque no encuentra su cierre, se reporta como error léxico y se descarta el resto del archivo, dado que no existe ningún carácter que garantice de forma no ambigua dónde debería haber terminado.

Si el carácter es un guion (-), se examina el siguiente carácter mediante el método `espiar()`; si corresponde a un mayor que (>), se reconoce el conector ->; en caso contrario, se reporta un error de carácter no reconocido.

Si el carácter corresponde a uno de los símbolos de un solo carácter del lenguaje (=, +, {, }, (,), ,), se invoca `leerSimboloSuelto()`, que genera el token correspondiente clasificándolo como operador o delimitador según cuál haya sido el símbolo.

Los espacios en blanco y saltos de línea se omiten sin generar ningún token, actualizando únicamente la fila y columna actuales mediante `avanzar()`.

Cualquier carácter que no encaje en ninguna de las categorías anteriores se reporta como un error léxico de carácter no reconocido, sin detener el análisis del resto del archivo.

Manejo de Errores Léxicos

El analizador contempla tres tipos de error léxico:

Carácter no reconocido: ocurre cuando el analizador encuentra un símbolo que no corresponde a ninguna categoría válida del lenguaje (por ejemplo, `#`, `$`, `%`). En este caso, se registra un error con el carácter problemático, su fila y columna, y el análisis continúa avanzando a partir del siguiente carácter. Este es el caso de recuperación más económico: se descarta un único carácter y se retoma el análisis de inmediato.

Cadena sin cerrar: ocurre cuando se encuentra una comilla de apertura (`"`) pero el analizador llega a un salto de línea o al final del archivo sin haber encontrado la comilla de cierre correspondiente. Se registra un error con el contenido leído hasta el momento del fallo, la fila y la columna donde comenzó la cadena. Cabe aclarar que, dado que el analizador reconoce el texto de forma determinista y sin memoria de lo ocurrido previamente en el archivo, cualquier comilla suelta se interpreta siempre como una apertura; no es posible distinguir, desde el punto en que se detecta el error, si se trataba de una cadena verdaderamente sin abrir o sin cerrar.

Comentario de bloque sin cerrar: ocurre cuando se encuentra la secuencia de apertura `/*` pero el analizador llega al final del archivo sin encontrar la secuencia de cierre `*/`. A diferencia de los dos casos anteriores, aquí no existe ningún carácter o símbolo del lenguaje que sirva como límite no ambiguo para detener la búsqueda del cierre, dado que un comentario de bloque válido puede extenderse legítimamente por varias líneas y contener cualquier carácter, incluidos saltos de línea. Por esta razón, la única recuperación posible es reportar un único error y descartar el resto del archivo como parte del comentario, comportamiento equivalente al adoptado por compiladores de propósito general ante la misma situación.

En todos los casos, el análisis nunca se detiene ante un error; el recorrido continúa hasta procesar la totalidad del archivo, acumulando todos los errores encontrados en una lista independiente de la lista de tokens reconocidos, lo que permite reportar en una sola ejecución la totalidad de los errores léxicos presentes.

Pseudocódigo de métodos importantes

Método espiar()

espiar()

Si posicion + 1 es menor que la longitud del texto entonces

retornar el caracter en la posicion + 1

Si no

retornar caracter nulo

Fin espiar

Método avanzar()

avanzar()

Si el caracter en la posicion actual es salto de linea entonces

incrementar fila

columna = 1

Si no

incrementar columna

incrementar posicion

Fin avanzar

Método leerCadena()

leerCadena()

guardar fila y columna de inicio

avanzar() // se salta la comilla de apertura

cerrada = falso

Mientras haya texto por leer hacer

Si el caracter actual es comilla entonces

avanzar() // se salta la comilla de cierre

cerrada = verdadero

salir del ciclo

Si el caracter actual es salto de linea entonces

salir del ciclo

acumular el caracter actual

avanzar()

FinMientras

Si cerrada es verdadero entonces

crear Token de tipo CADENA con lo acumulado

Si no

crear ErrorLexico "Cadena sin cerrar"

Fin leerCadena

Método leerNumero()

```
leerNumero()
    guardar fila y columna de inicio
    esDecimal = falso
    Mientras haya texto por leer hacer
        Si el caracter actual es digito entonces
            acumular el caracter, avanzar()
        Si no, si el caracter actual es punto Y no esDecimal
            Y el siguiente caracter (espiado) es digito entonces
                esDecimal = verdadero
                acumular el punto, avanzar()
        Si no
            salir del ciclo
    FinMientras
    Si esDecimal entonces
        crear Token de tipo DECIMAL
    Si no
        crear Token de tipo ENTERO
    Fin leerNumero
```

Método analizar()

```
analizar()
    Mientras haya texto por leer hacer
        caracter_actual = obtener caracter en posicion actual
        Si es espacio o salto de linea entonces
            avanzar()
        Si no, si es letra o guion bajo entonces
            leerPalabra()
        Si no, si es arroba entonces
            leerDirectiva()
        Si no, si es comilla entonces
            leerCadena()
        Si no, si es digito entonces
            leerNumero()
        Si no, si es diagonal entonces
            leerComentario()
        Si no, si es guion entonces
            leerConectorFlecha()
        Si no, si es simbolo suelto valido entonces
            leerSimboloSuelto()
        Si no
            crear ErrorLexico "Caracter no reconocido"
            avanzar()
```

FinMientras
Fin analizar

Método leerComentario()

```
leerComentario()  
  avanzar() // se salta la primera diagonal  
  siguiente = caracter actual  
  
  Si siguiente es diagonal entonces  
    // comentario de linea  
    avanzar()  
    Mientras haya texto y el caracter actual no sea salto de linea hacer  
      avanzar()  
    FinMientras  
  Si no, si siguiente es asterisco entonces  
    // comentario de bloque  
    avanzar()  
    cerrado = falso  
    Mientras haya texto por leer hacer  
      Si el caracter actual es asterisco Y el siguiente (espiado) es diagonal entonces  
        avanzar() // salta el asterisco  
        avanzar() // salta la diagonal  
        cerrado = verdadero  
        salir del ciclo  
      avanzar()  
    FinMientras  
    Si cerrado es falso entonces  
      crear ErrorLexico "Comentario de bloque sin cerrar"  
Fin leerComentario
```

Método leerConectorFlecha

```
leerConectorFlecha()  
  guardar fila y columna de inicio  
  Si el siguiente caracter (espiado) es mayor-que entonces  
    avanzar() // salta el guion  
    avanzar() // salta el mayor-que  
    crear Token de tipo CONECTOR con lexema "->"  
  Si no  
    crear ErrorLexico "Caracter no reconocido" con lexema "-"  
    avanzar()  
Fin leerConectorFlecha
```

Método generarCodigoDOT() (estructura resumida)

generarCodigoDOT()

- iniciar cadena DOT con encabezado del grafo (rankdir, nodo inicial)

- Para cada una de las siete ramas del AFD (símbolo suelto, flecha, número, cadena, directiva, comentario) hacer

 - agregar la definición de sus estados (con su forma y color

 - según sea tránsito, aceptación con token, aceptación sin token, o error)

 - agregar las transiciones correspondientes con su etiqueta

- FinPara

- cerrar la cadena DOT

- retornar la cadena completa

Fin generarCodigoDOT

Conclusión

El desarrollo del Proyecto 1 permitió elevar el tokenizador exploratorio construido en la Práctica 1 a un analizador léxico formalmente fundamentado. El diseño explícito de un autómata finito determinista para cada categoría de token, junto con su gramática regular equivalente, permitió verificar que la implementación manual del analizador correspondiera fielmente al diseño teórico planteado, en lugar de construirse únicamente por prueba y error.

El diseño del recorrido carácter por carácter, junto con la técnica de anticipación (lookahead) para casos como el conector `->` y la diferenciación entre comentarios de línea y de bloque, permitió construir un tokenizador robusto capaz de clasificar correctamente las diez categorías de tokens definidas por el lenguaje.

La implementación del manejo de errores léxicos, capaz de continuar el análisis tras encontrar un error sin detener el procesamiento del resto del archivo, refuerza la utilidad práctica del programa como una herramienta que informa de manera completa sobre la validez de un archivo de entrada. Particularmente, el análisis de los tres tipos de error contemplados evidenció los límites estructurales propios de un reconocedor determinista de una sola pasada, como la imposibilidad de distinguir una cadena sin abrir de una sin cerrar, o la necesidad de descartar el resto del archivo ante un comentario de bloque sin cierre.

La incorporación de una interfaz gráfica desarrollada en Java Swing, junto con la generación automática de reportes HTML y la graficación del autómata mediante Graphviz a partir de código DOT emitido por el propio programa, transformó el analizador de una herramienta de consola en una aplicación completa, operable sin necesidad de conocimiento técnico previo por parte del usuario final.

Como resultado final se obtuvo un analizador léxico funcional y formalmente respaldado, acompañado de su diseño teórico (AFD y gramática regular), una interfaz gráfica completa, reportes exportables a formato HTML, y la visualización del autómata, cumpliendo con la totalidad de los requerimientos establecidos para esta entrega del proyecto.